

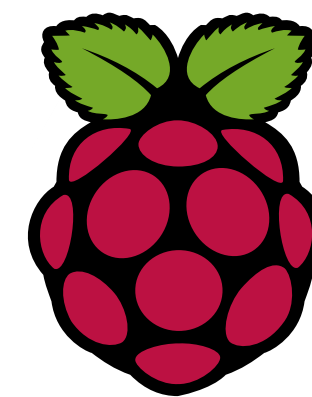
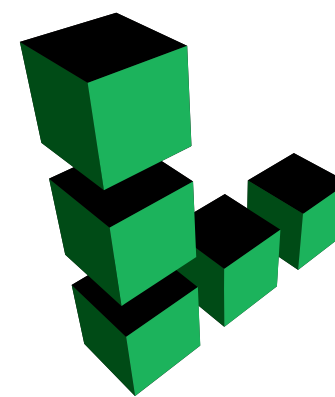
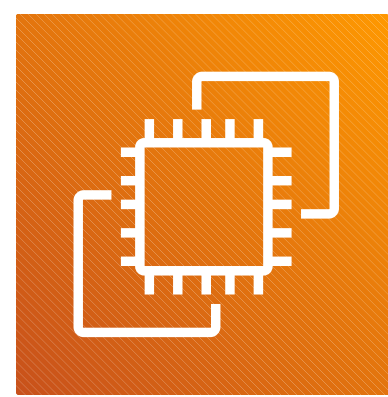
# **Module 1: The foundation**

## **Cloud setup & access**

# Cloud setup & access

## Prerequisites

- **Goal:** A secure environment before deploying code.
- **Providers:** DigitalOcean, AWS EC2, Linode, or Raspberry Pi.



# Cloud setup & access

## Prerequisites

- **Goal:** A secure environment before deploying code.
- **Providers:** DigitalOcean, AWS EC2, Linode, or Raspberry Pi.

## Learning objectives

- Deploy a server running **Ubuntu 24.04 LTS**.
- Generate and configure **SSH Keys** ([Ed25519](#)).
- Understand the "**Lock & Key**" security model.
- **Security:** Disable password authentication completely.

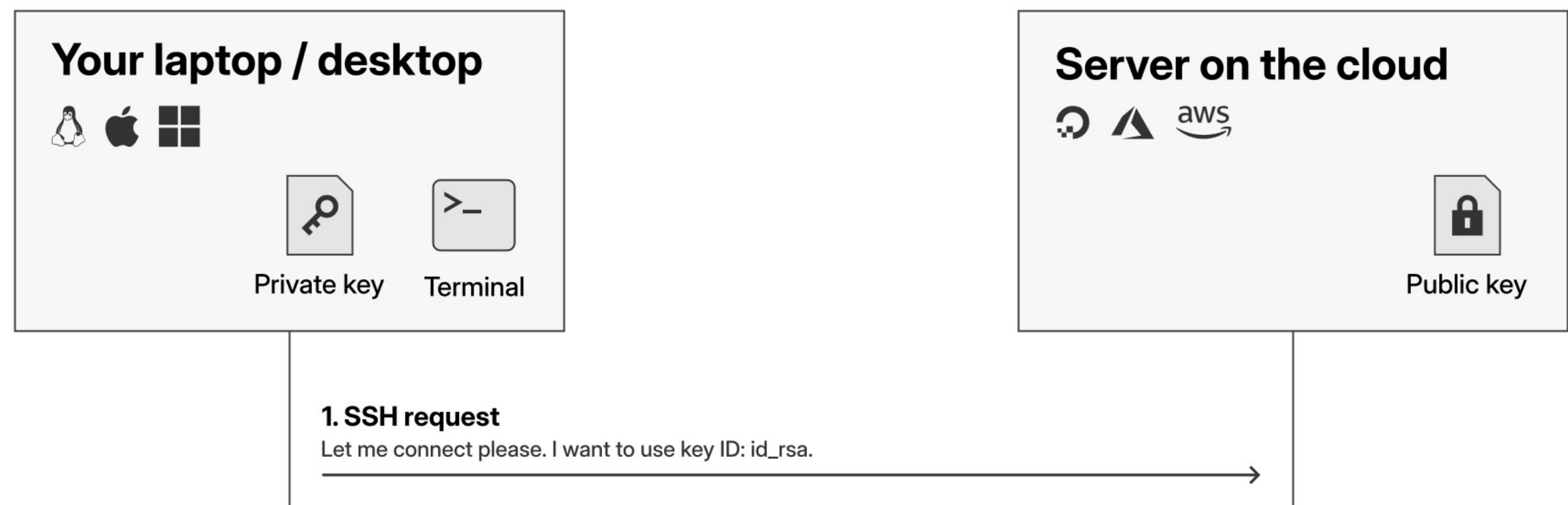


# How SSH keys work

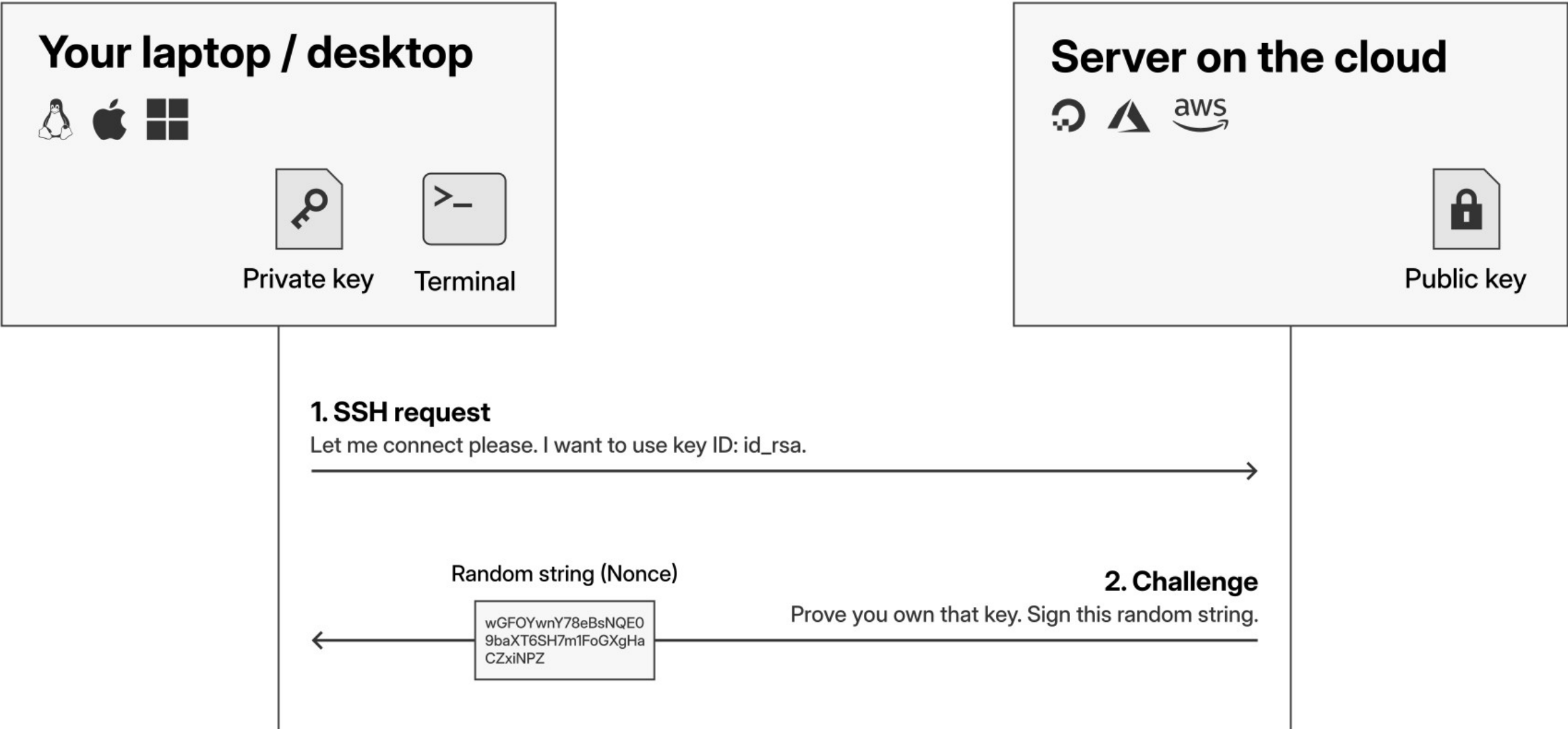




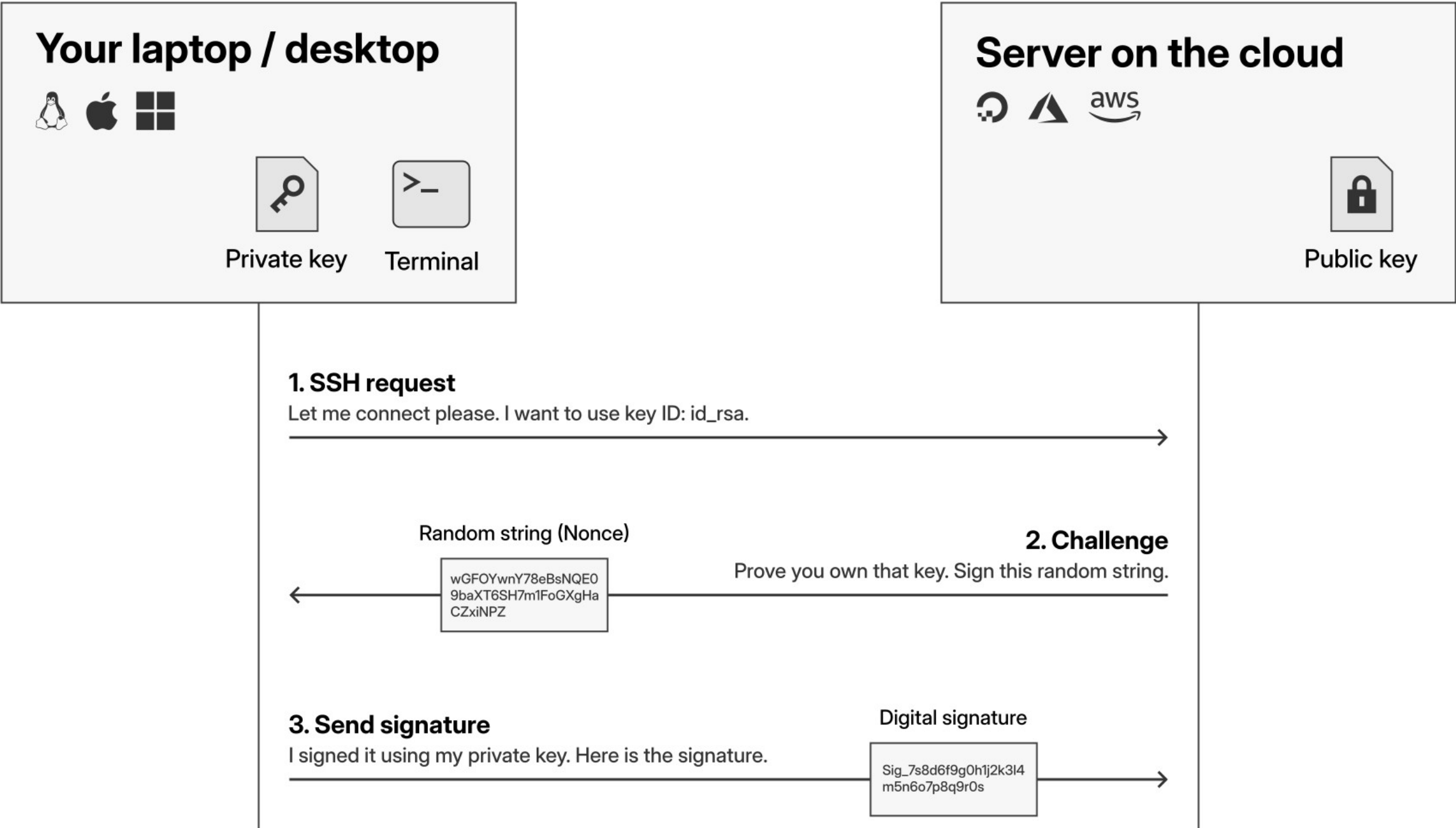
# How SSH keys work



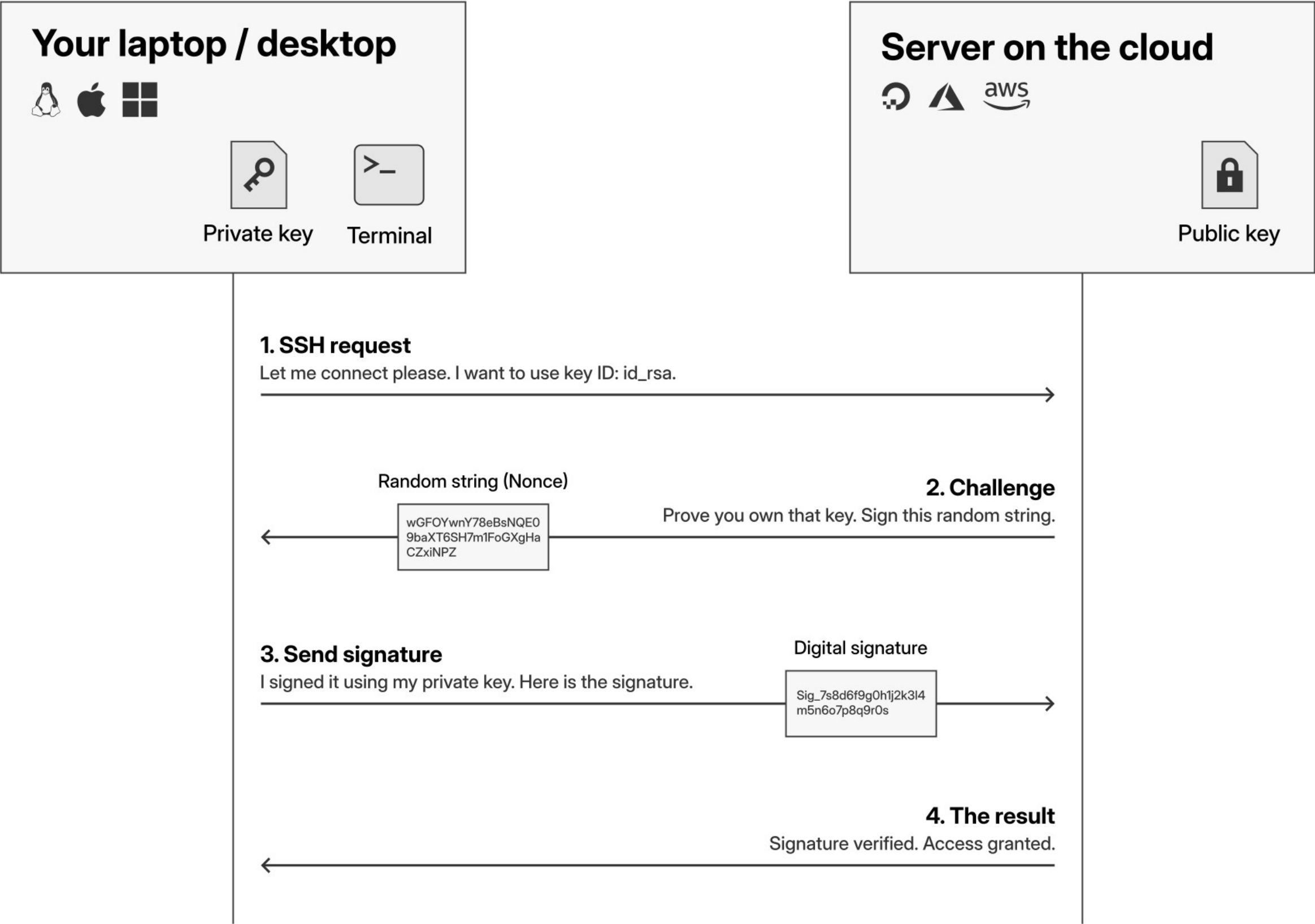
# How SSH keys work



# How SSH keys work



# How SSH keys work





# What is next?

- **Current status:** You now have a clean, secure Ubuntu server.
- **The risk:** The server is still exposed to the open internet.
- **Next steps:**
  - **Firewall configuration:** Setting up UFW (Uncomplicated Firewall).
  - **Brute force protection:** Installing Fail2Ban to block attackers.
  - **Emergency access:** How to use the recovery console.

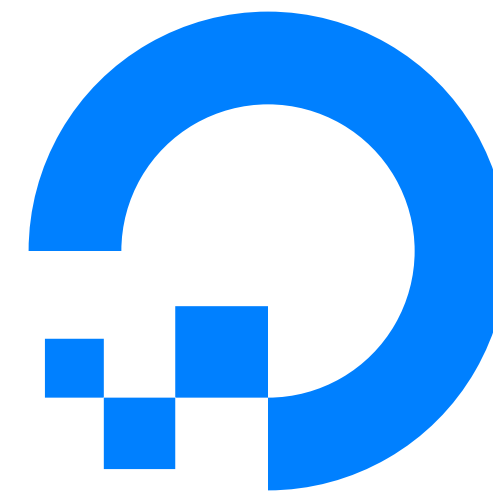


# **Module 1: The foundation**

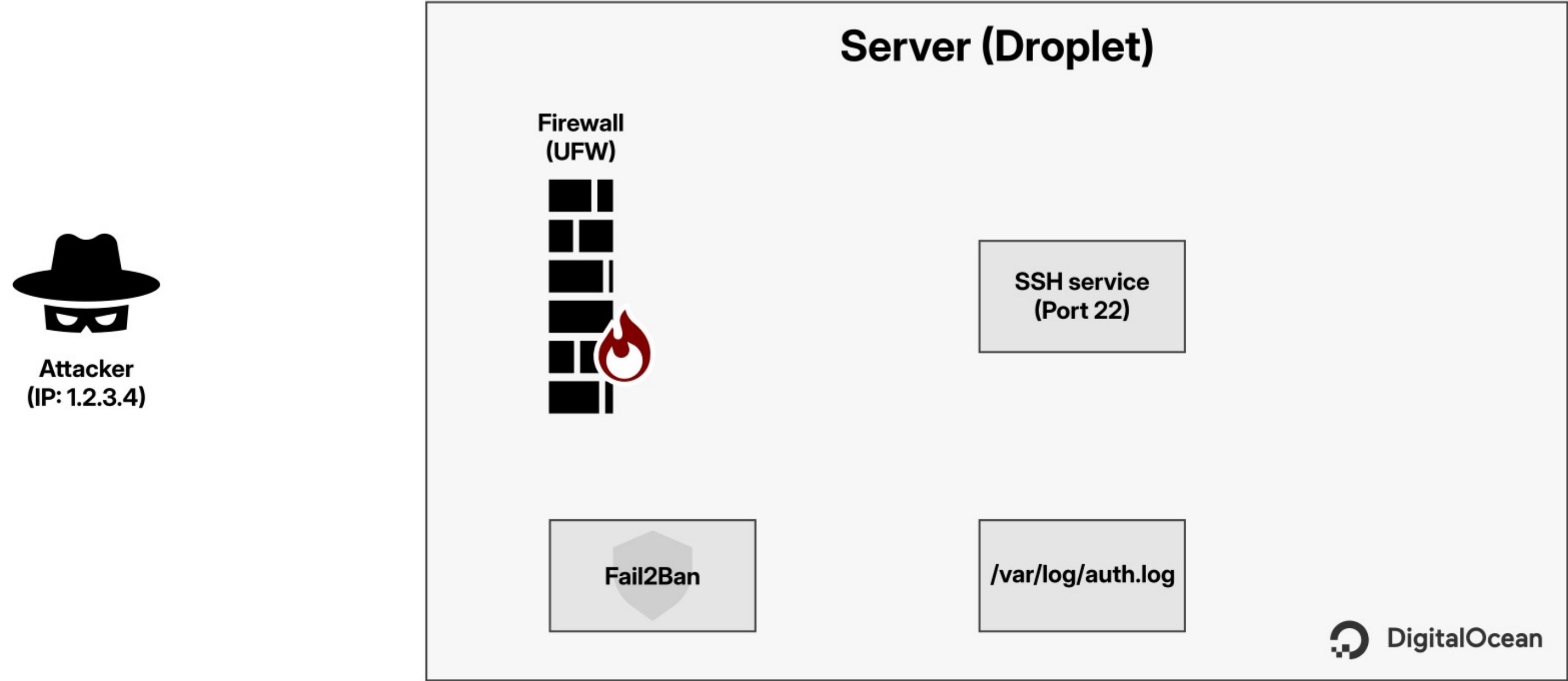
## **The firewall strategy**

# The firewall strategy

- **Problem:** Default Linux servers accept traffic on all ports.
- **Solution:** Use UFW to block everything by default.
- **Automation:** Use Fail2Ban to block bots instantly.
- **Safety net:** Using the DigitalOcean Recovery Console.

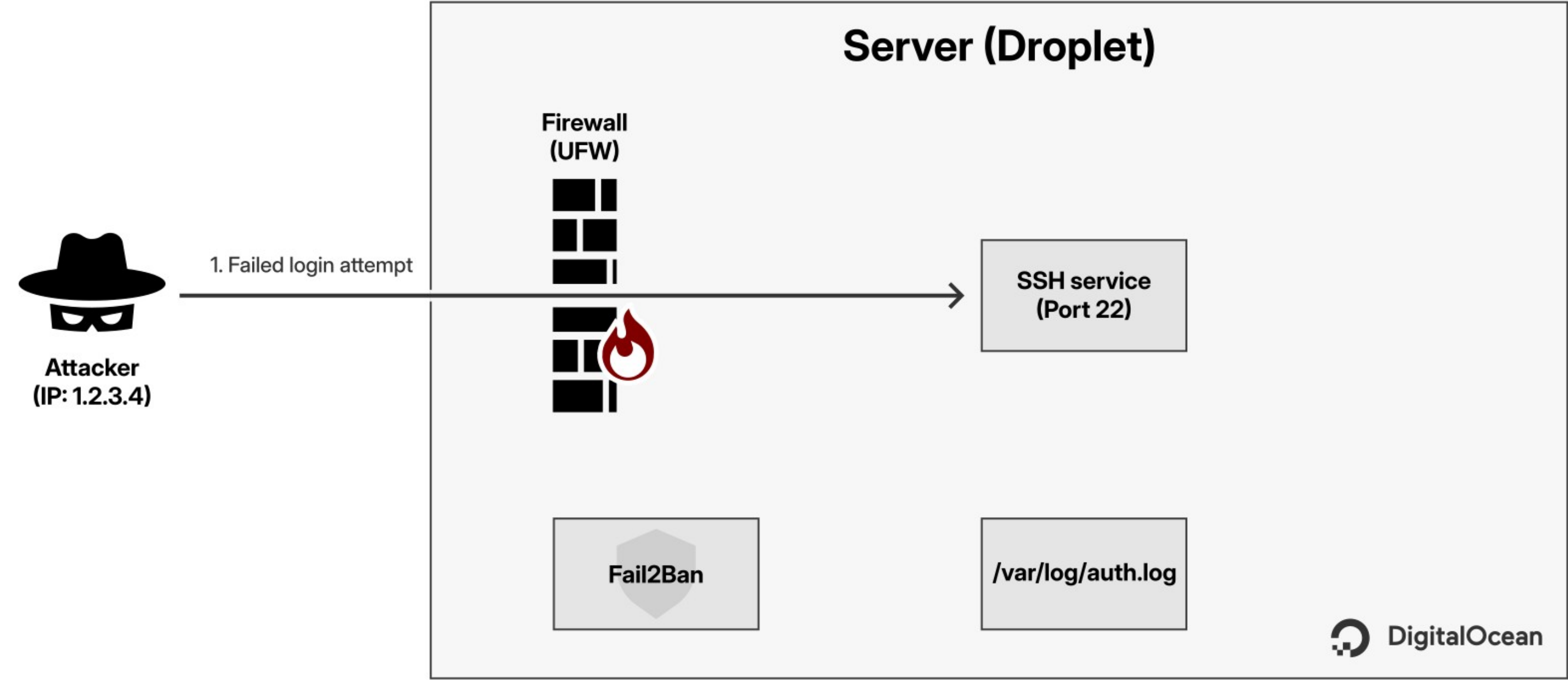


# How Fail2Ban works

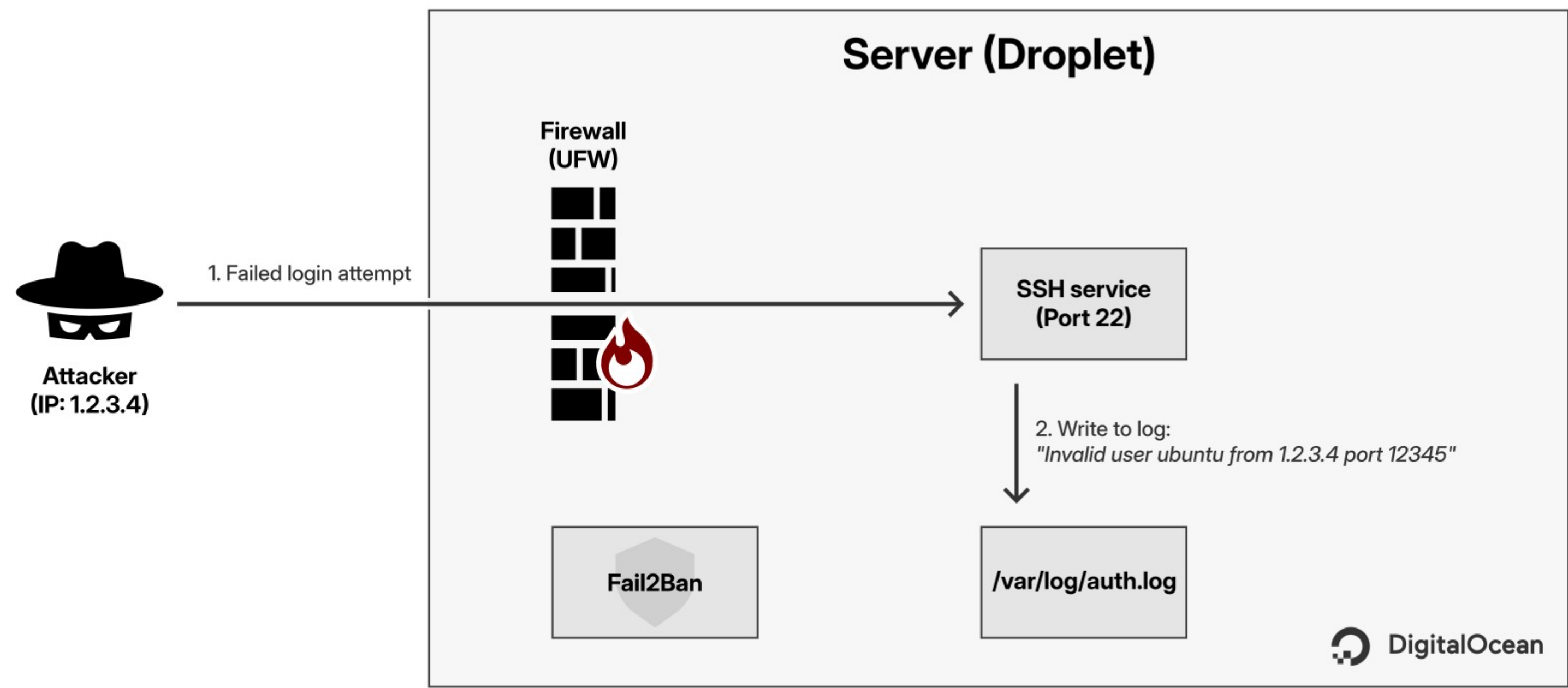




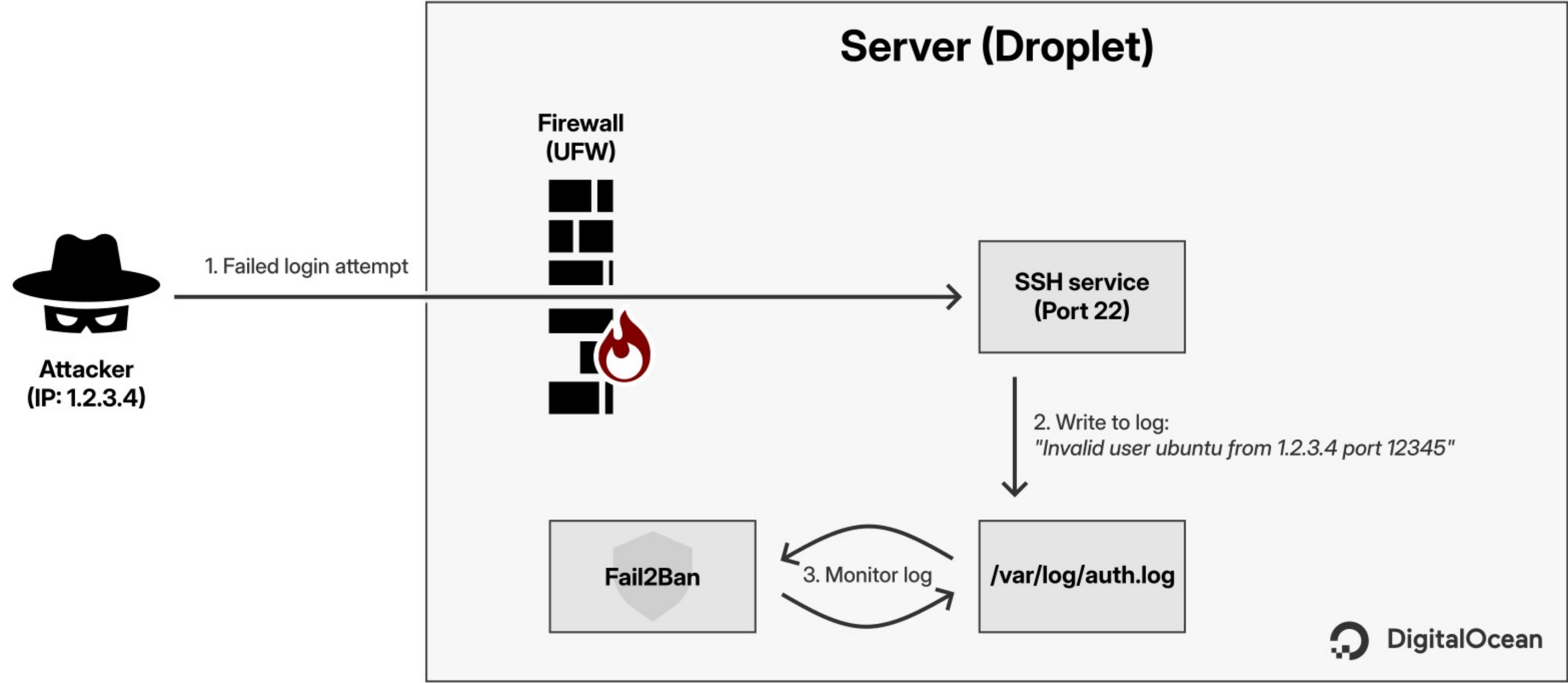
# How Fail2Ban works



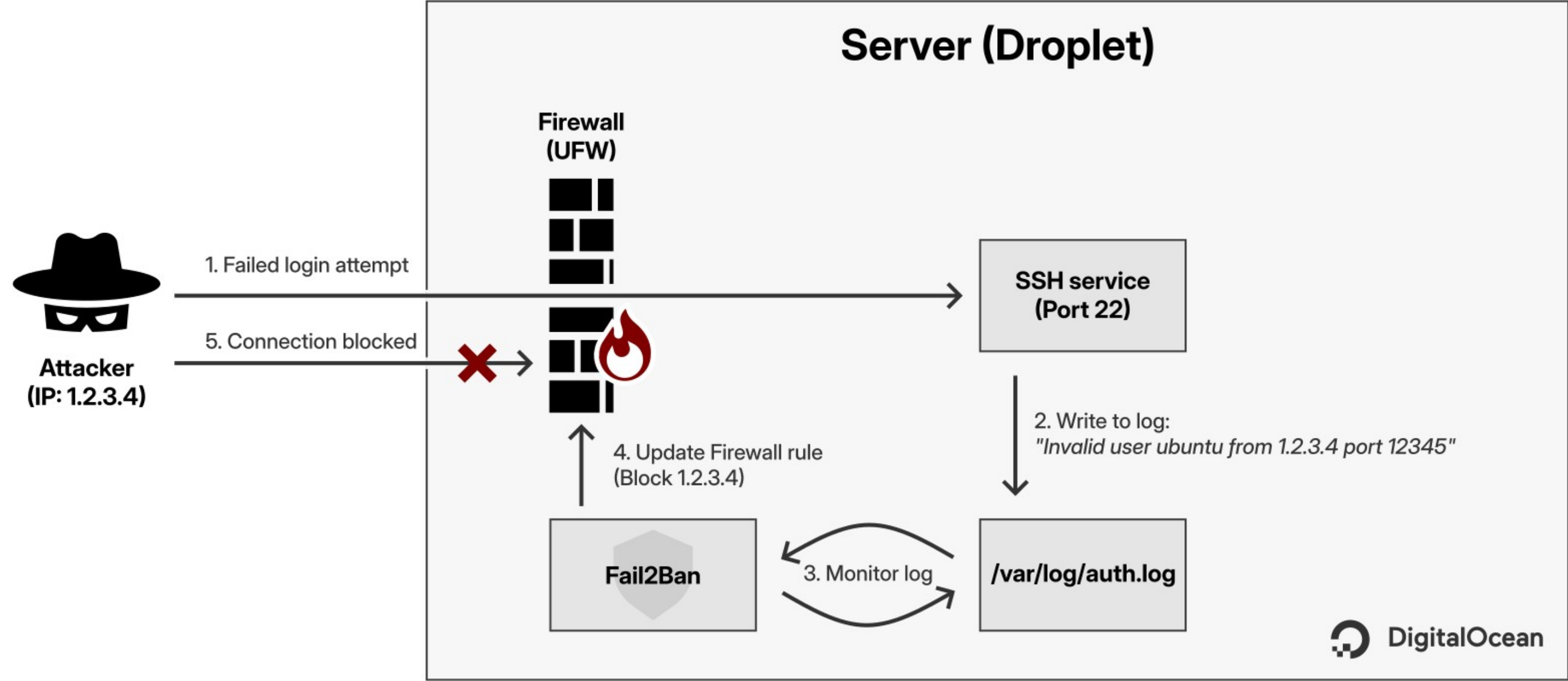
# How Fail2Ban works



# How Fail2Ban works



# How Fail2Ban works



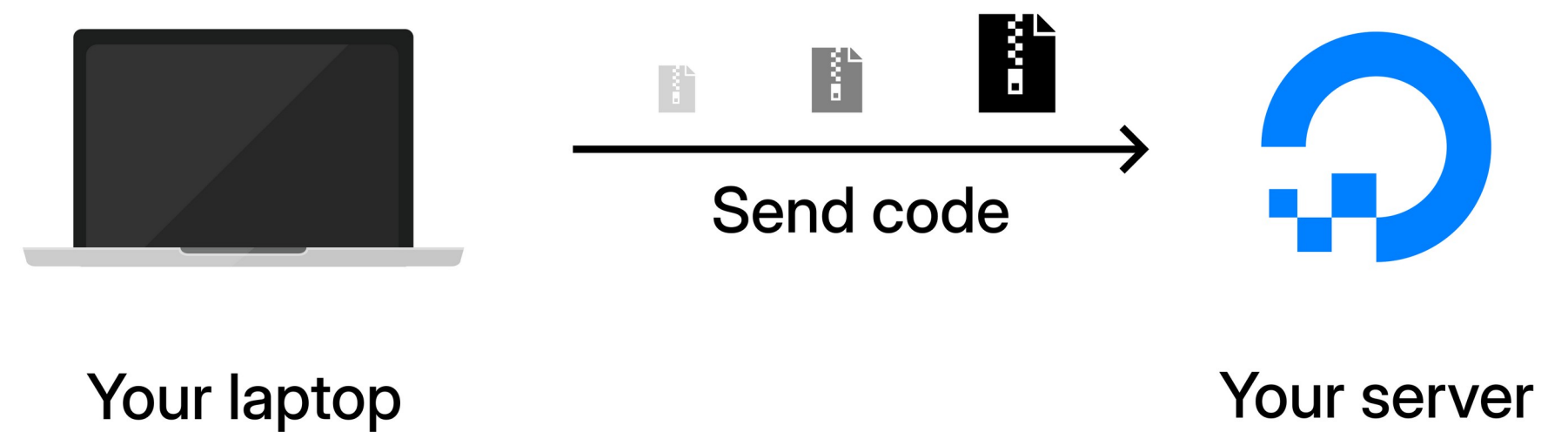


# **Module 2: The application runtime**

## **Manual deployment**

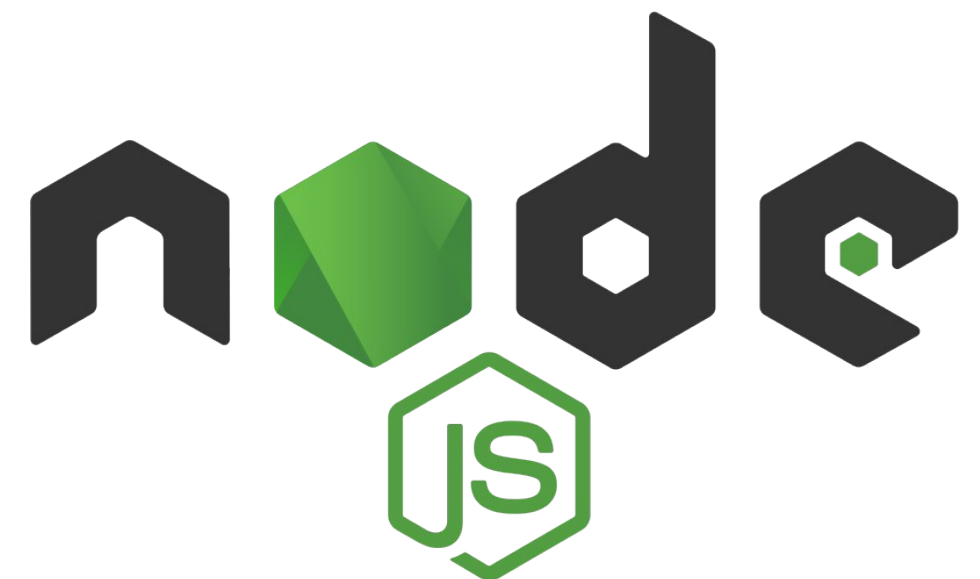
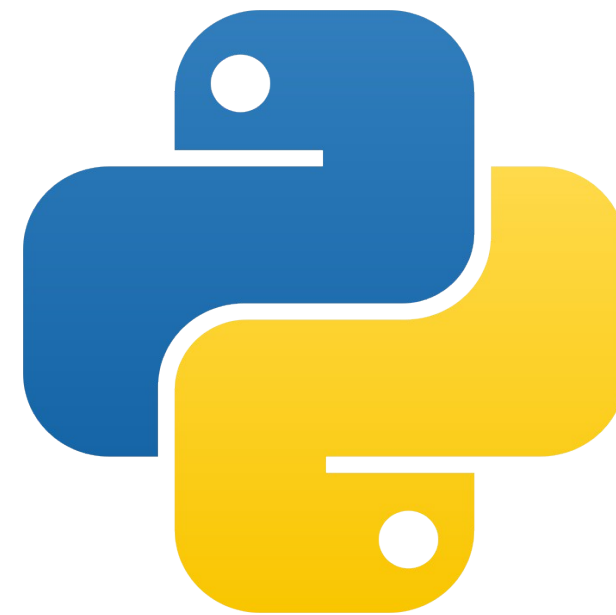
# Manual deployment

- Move code using **scp** and organize permissions.



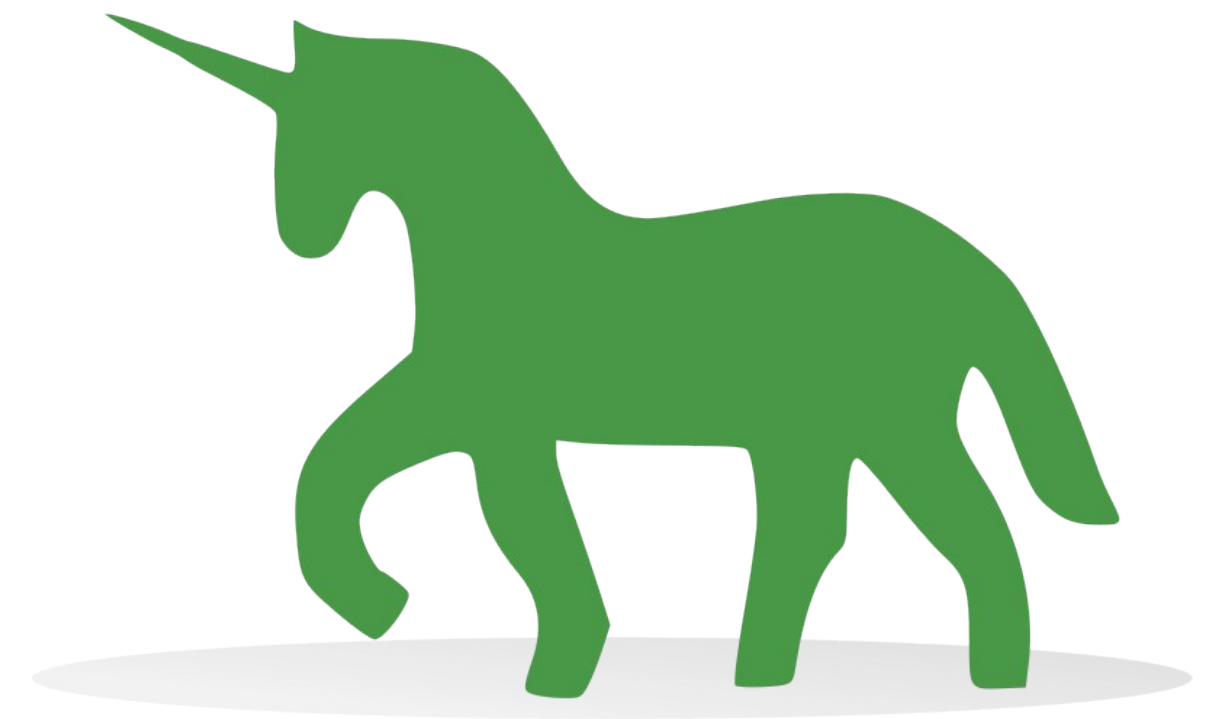
# Manual deployment

- Move code using **scp** and organize permissions.
- Install **Python** (Backend) and **Node.js** (Frontend).
- Fix build errors using **Swap files**.
- Use **SSH Tunnels** to bypass the **firewall**.



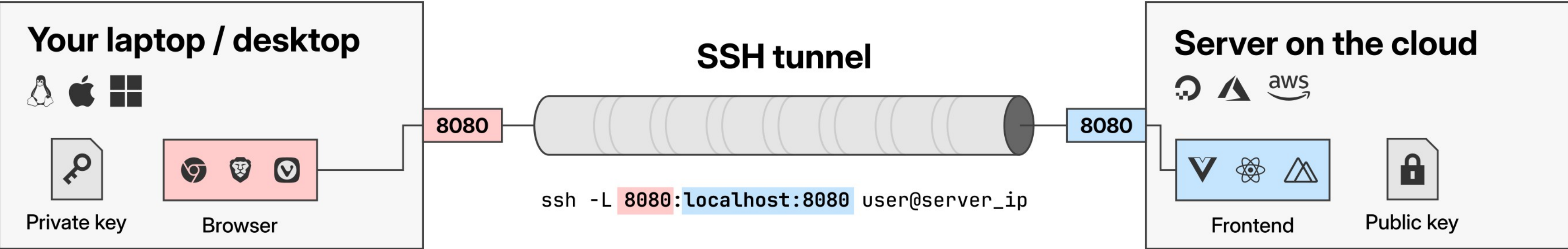
# Manual deployment

- Move code using **scp** and organize permissions.
- Install **Python** (Backend) and **Node.js** (Frontend).
- Fix build errors using **Swap files**.
- Use **SSH Tunnels** to bypass the **firewall**.
- Configure **Gunicorn** and **Supervisor**.

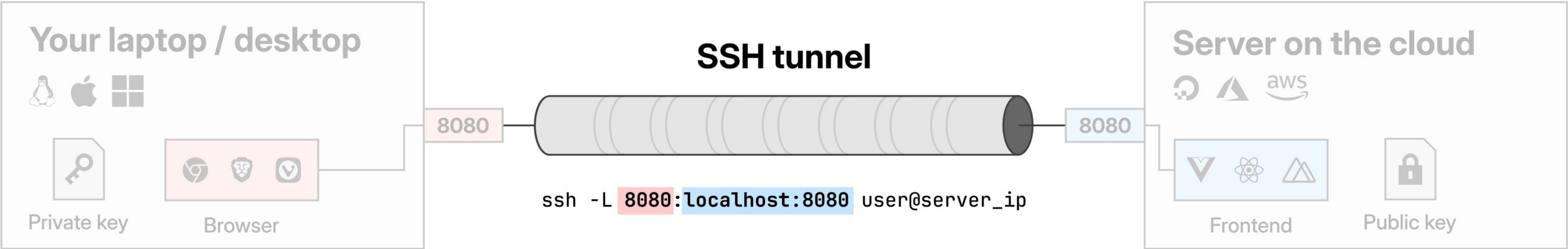




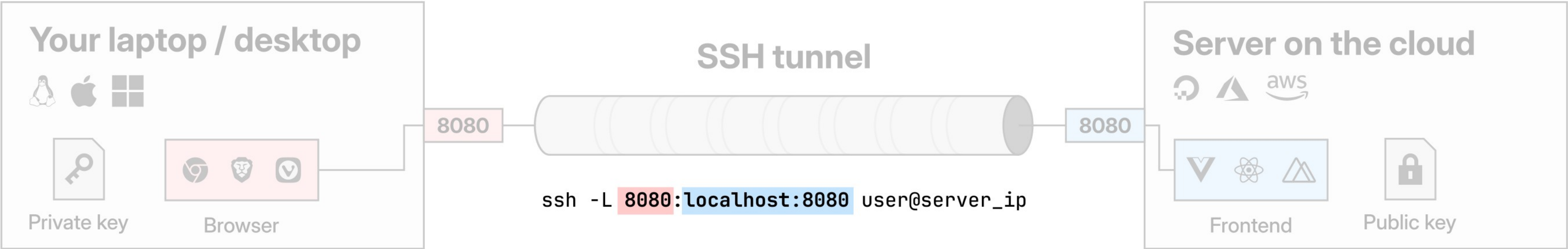
# SSH tunneling



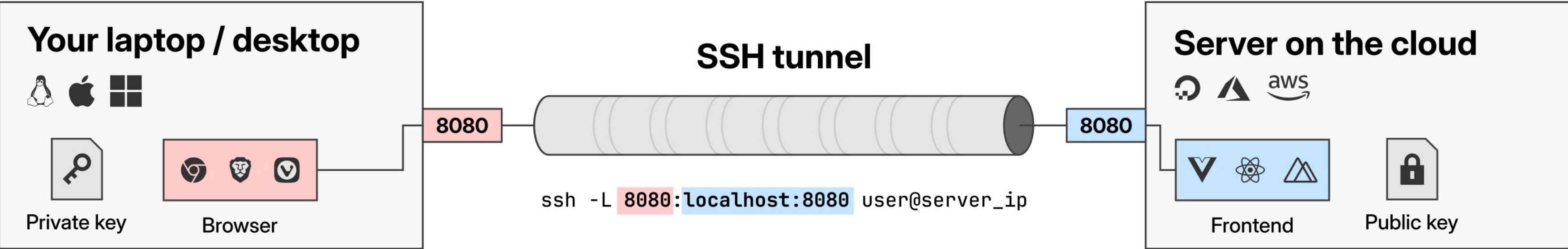
# SSH tunneling



# SSH tunneling

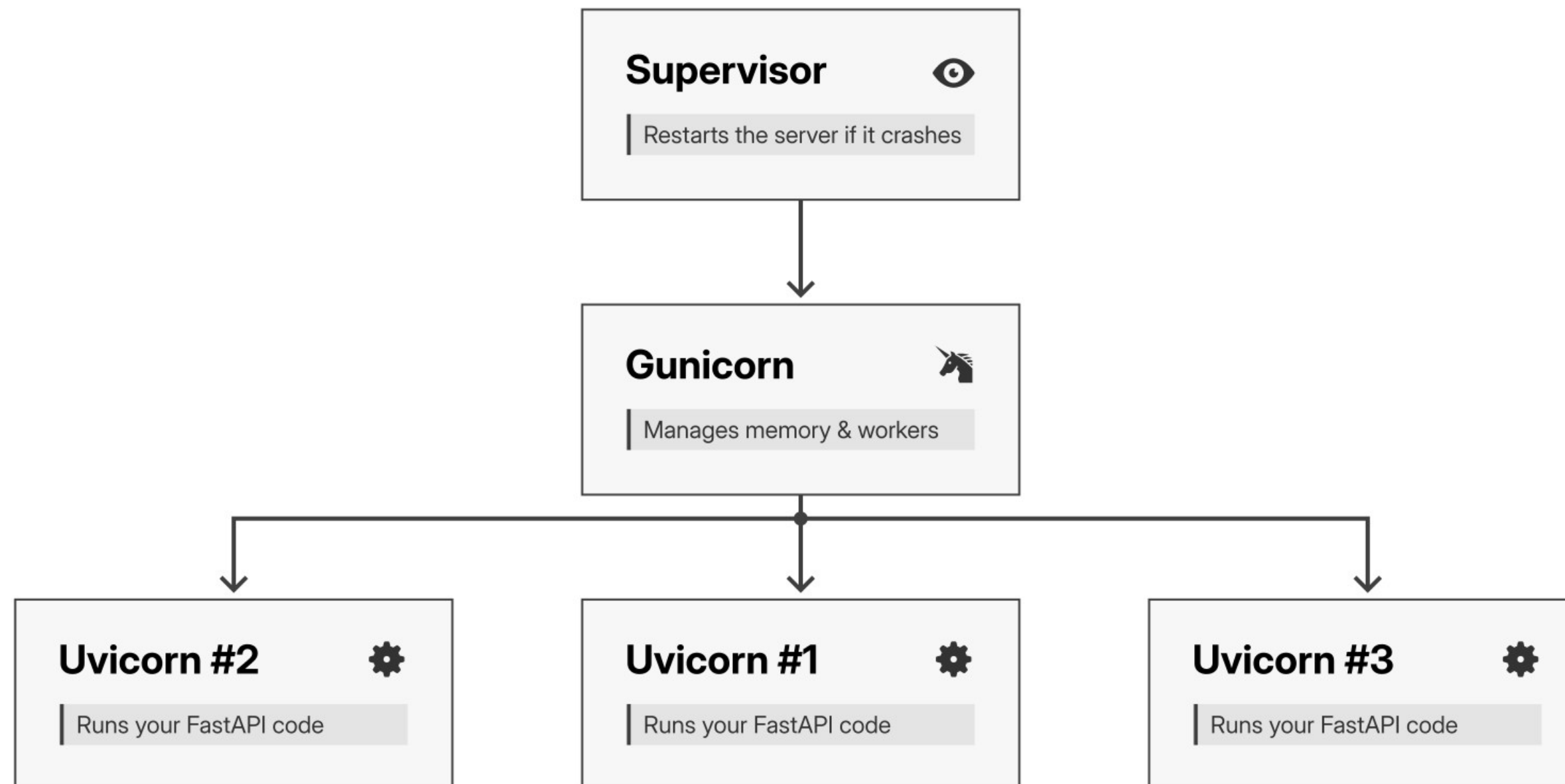


# SSH tunneling





# Keeping the app alive



# **Module 2: The application runtime**

**Reverse proxy & security headers**

# Reverse proxy & Security

- Sync frontend and backend URLs.
- Install and configure **Nginx**.



# Reverse proxy & Security

- Sync frontend and backend URLs.
- Install and configure **Nginx**.
- Understand and fix the **502 Bad Gateway** error.

## 502 Bad Gateway

---

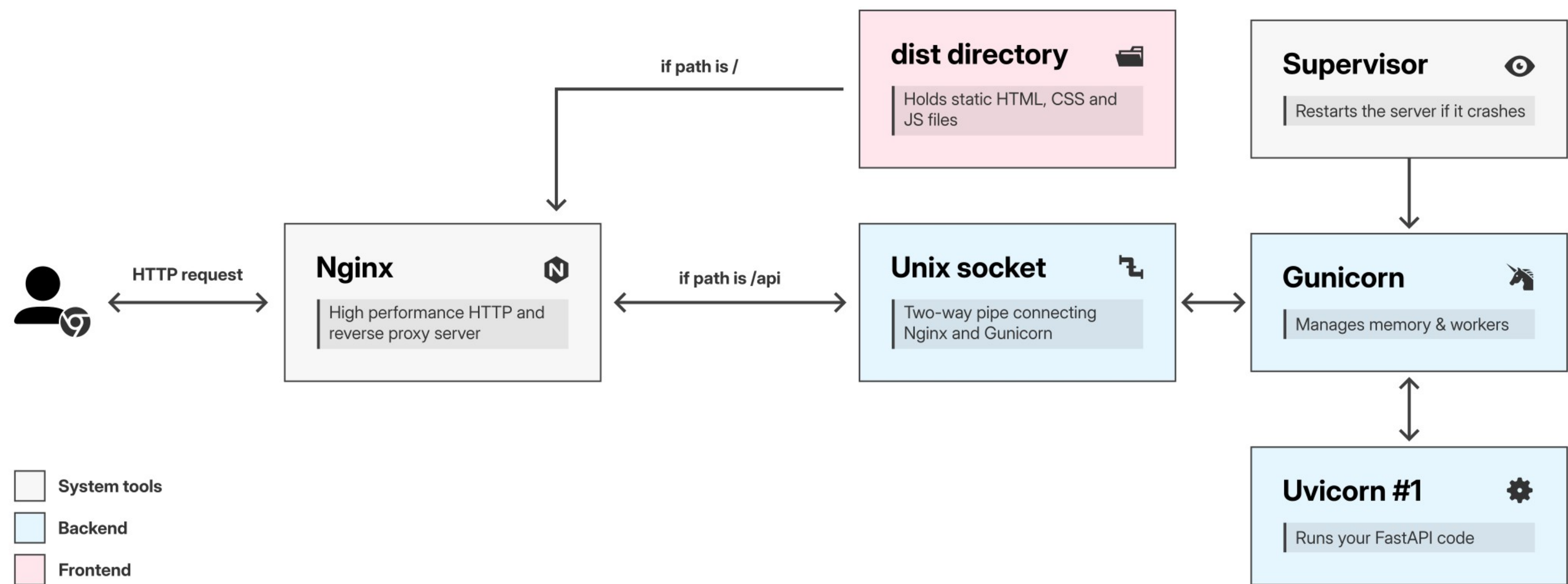
nginx/1.14.0 (Ubuntu)



# Reverse proxy & Security

- Sync frontend and backend URLs.
- Install and configure **Nginx**.
- Understand and fix the **502 Bad Gateway** error.
- Protect users with **HTTP Security Headers (CSP)**.

# Nginx: The traffic cop



# Module 3: Data and search

## Self-hosting Meilisearch

# Self-hosting a search engine

- **The engine:** Meilisearch (a lightning-fast Rust binary).
- **Security:** Isolate the database using a **system user**.
- **Automation:** Run it as a background service using **systemd**.
- **Maintenance:** Schedule automatic snapshot backups.
- **Management:** Use an **SSH Tunnel** to access a visual dashboard securely.

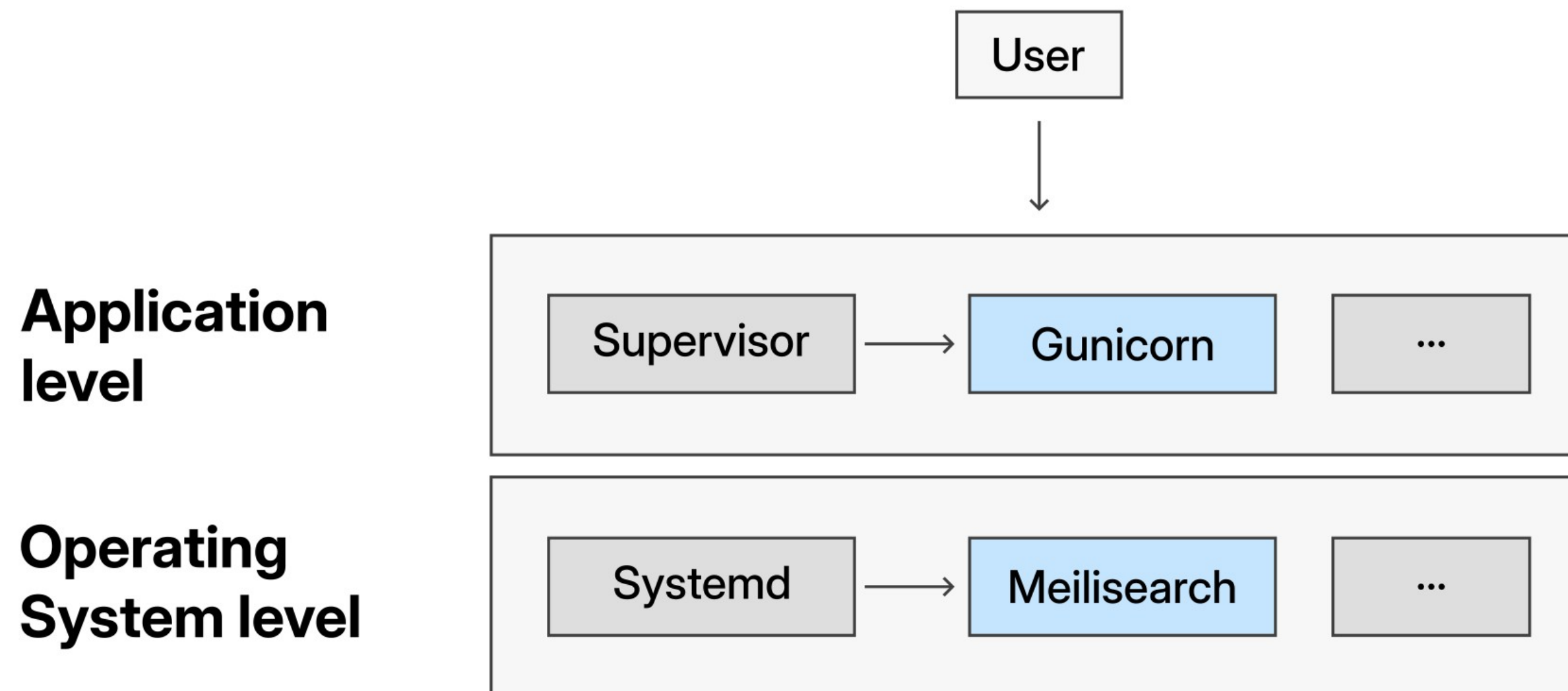




# What is a system user?

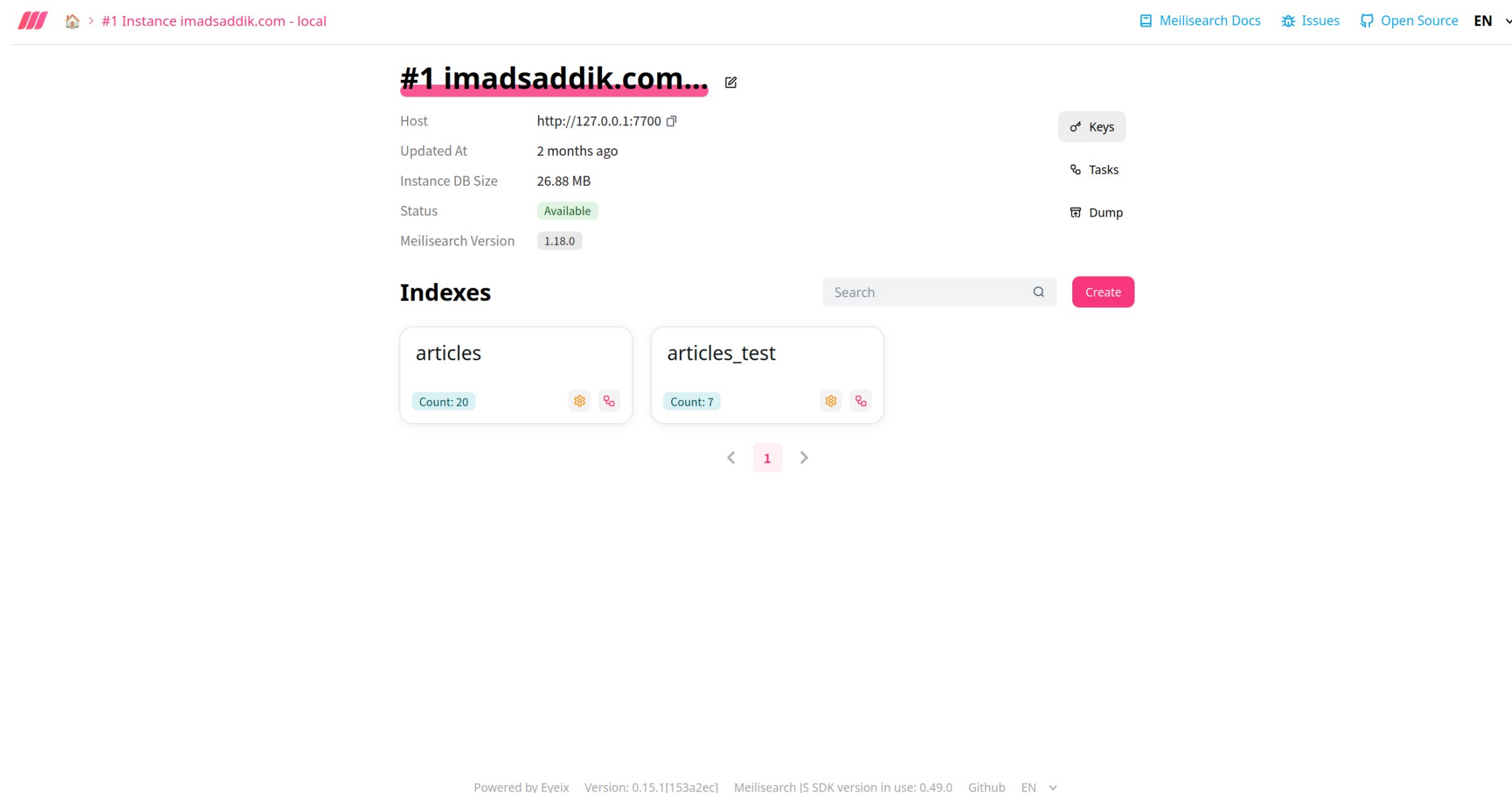
| Feature     | Standard user        | System user                 |
|-------------|----------------------|-----------------------------|
| Purpose     | Human administration | Running background services |
| Shell       | /bin/bash            | /bin/false                  |
| Login       | Yes                  | No                          |
| Home folder | /home/user           | /var/lib/meilisearch        |

# Systemd vs Supervisor



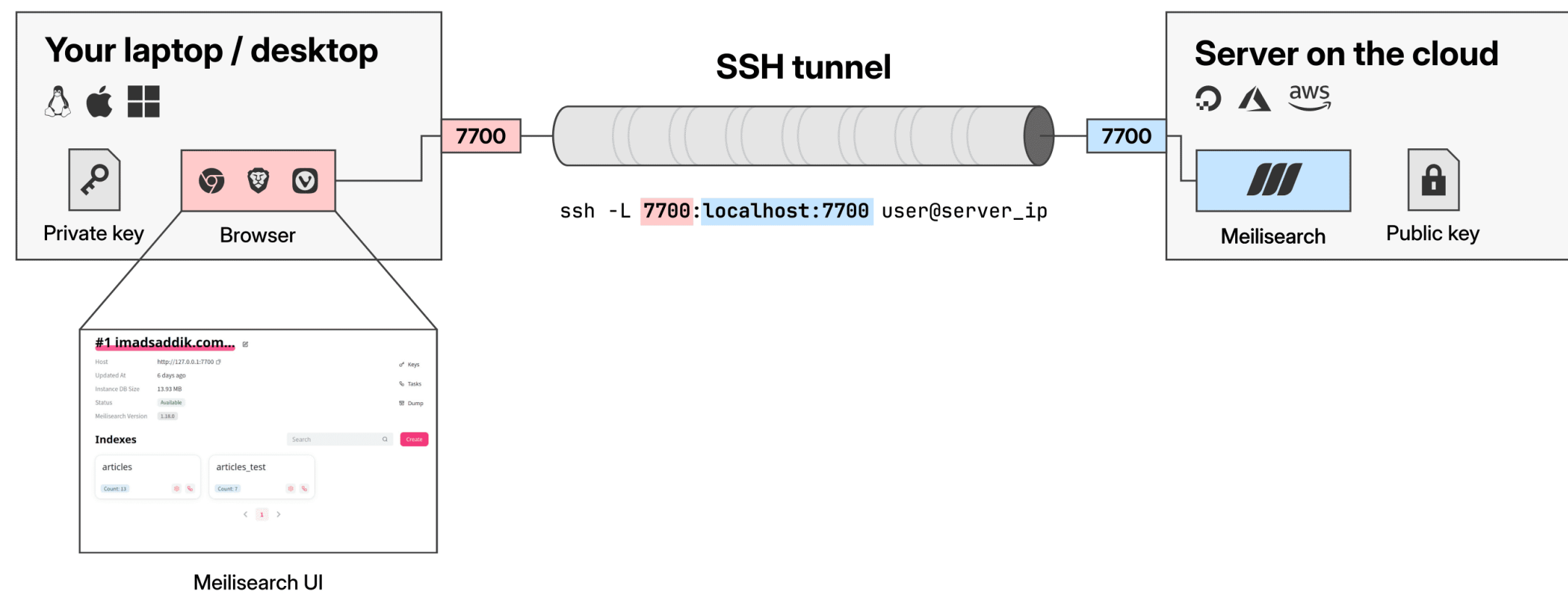
# Secure database management

- **The tool:** [meilisearch-ui](#) (Open-source web dashboard).
- **Features:** Search, add, and edit documents visually.



# Secure database management

- **The tool:** [meilisearch-ui](#) (Open-source web dashboard).
- **Features:** Search, add, and edit documents visually.
- **The connection:** Secure access via SSH Tunneling (no open ports).





# **Module 4: Global delivery and security**

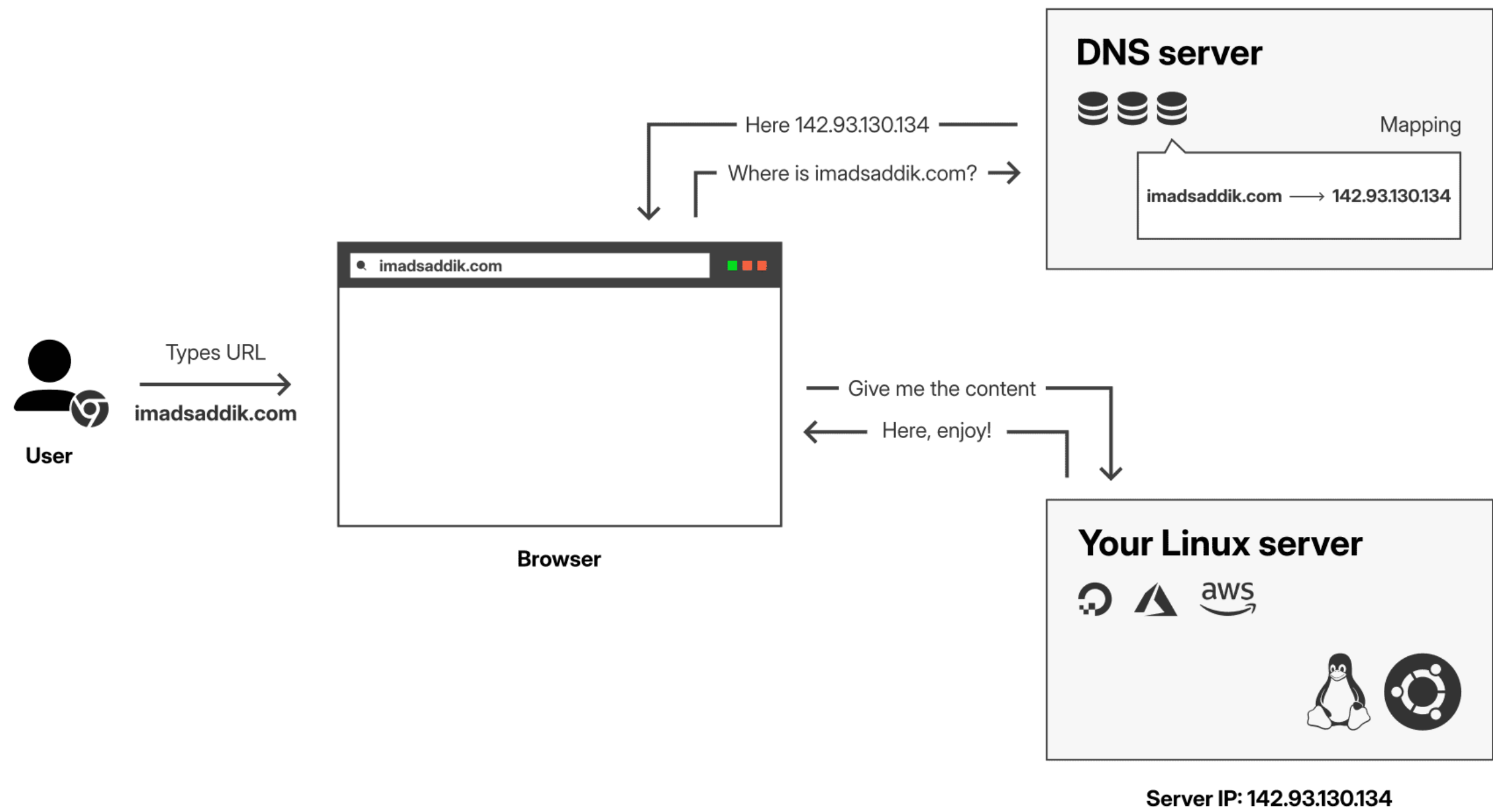
**Domains and SSL**

# Domains & SSL

- Purchase a **custom domain**.
- Configure **DNS Records** (A and CNAME).
- Install free **SSL certificates** via Let's Encrypt.
- Automate 90-day **renewals** using Cron.



# How DNS works



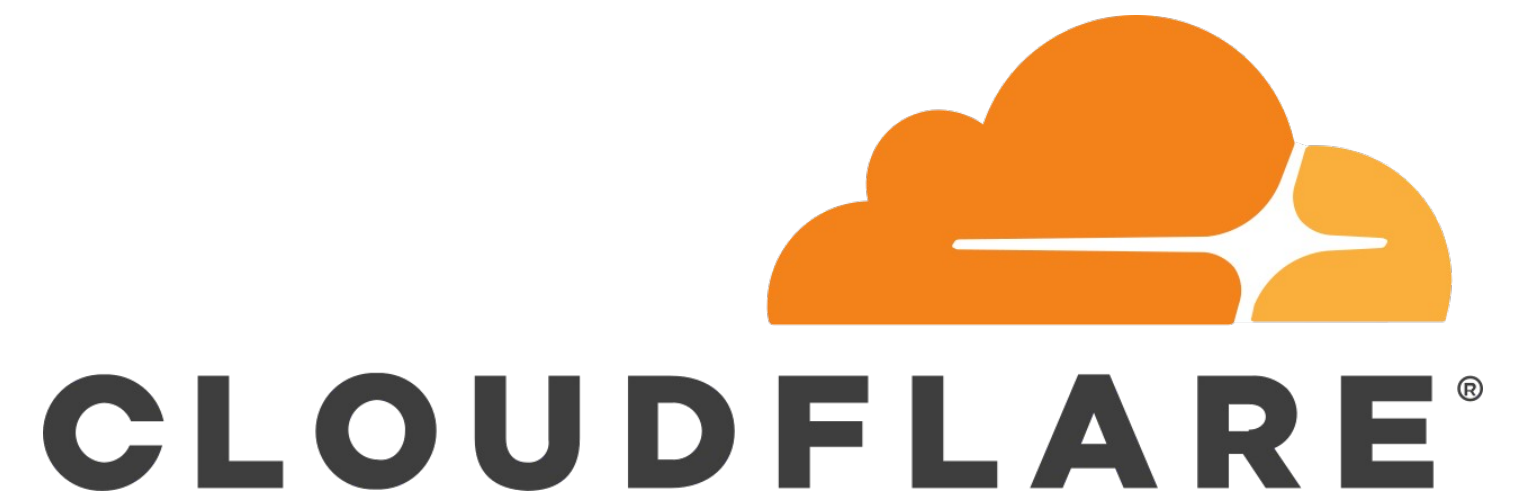
# **Module 4: Global delivery and security**

**The CDN layer and caching**



# Global speed & Caching

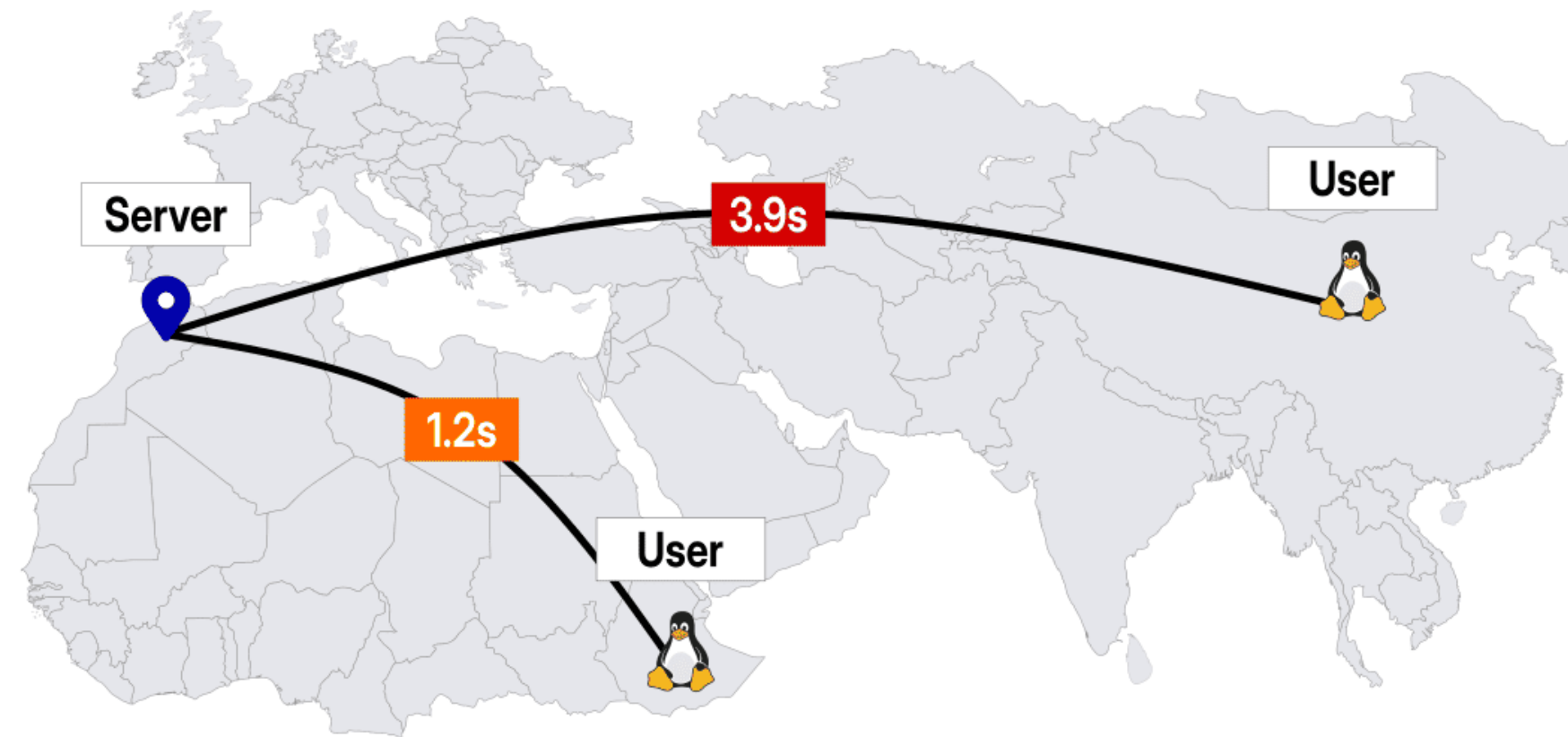
- Route traffic through a **Content Delivery Network (CDN)**.
- Configure strict **end-to-end SSL encryption**.
- Unmask visitor IP addresses in Nginx.
- Implement a **caching strategy** for SPAs.



# Beating the speed of light

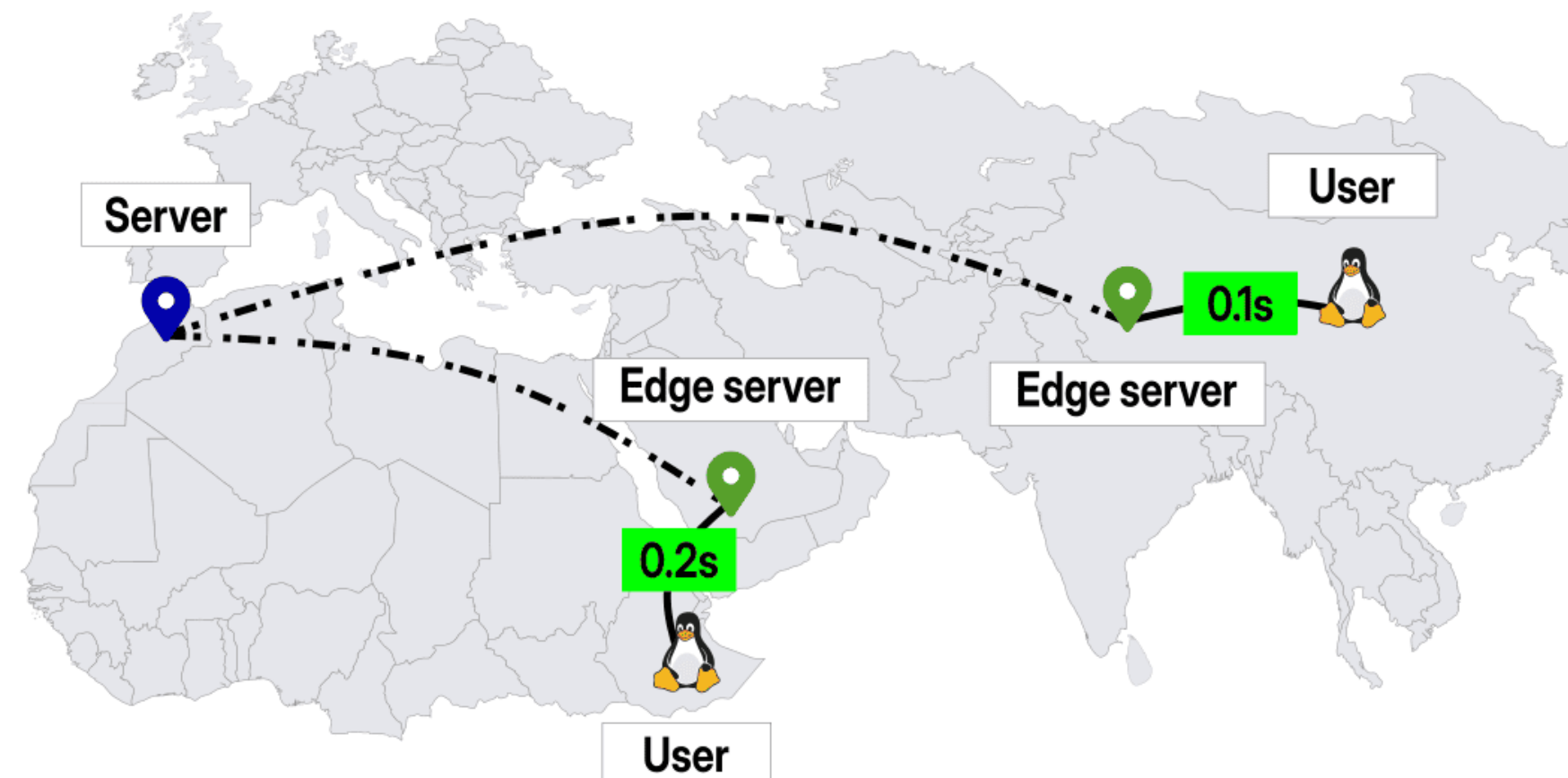
## Before CDN

Users connect directly to your origin server. Physical distance causes slow load times for international visitors.



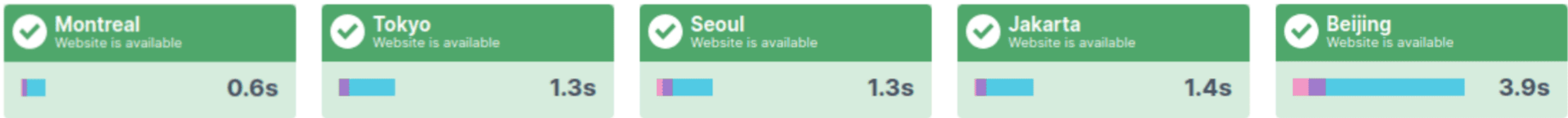
## After CDN

A global network of edge servers caches your files. Users connect to a server right next to them, resulting in lightning-fast load times.

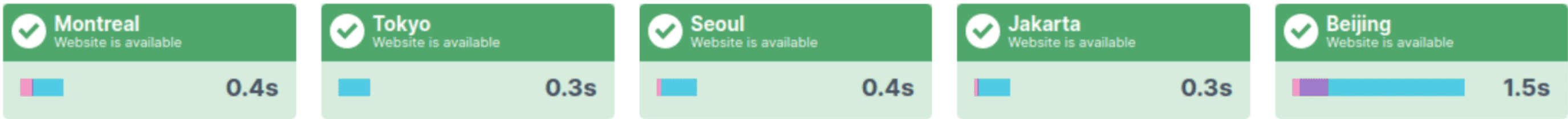


# Beating the speed of light

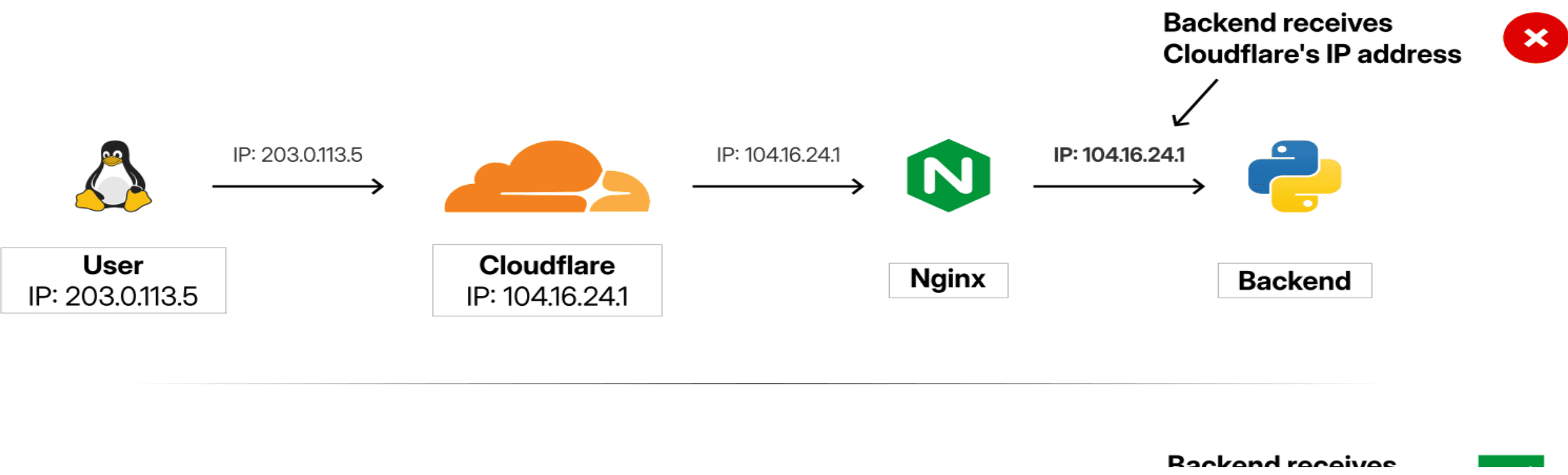
## Before using Cloudflare



## After using Cloudflare

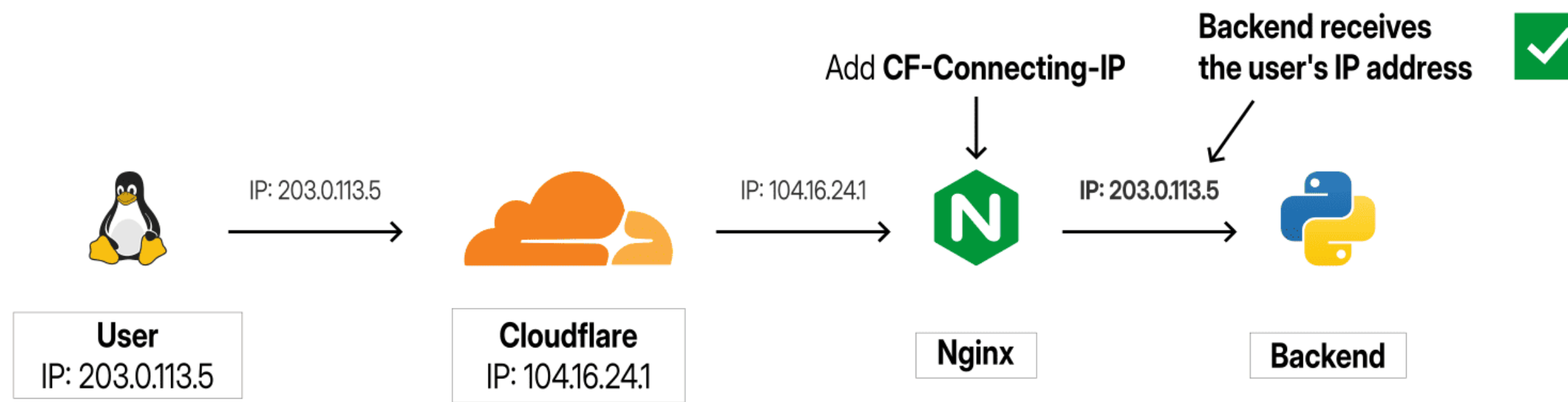
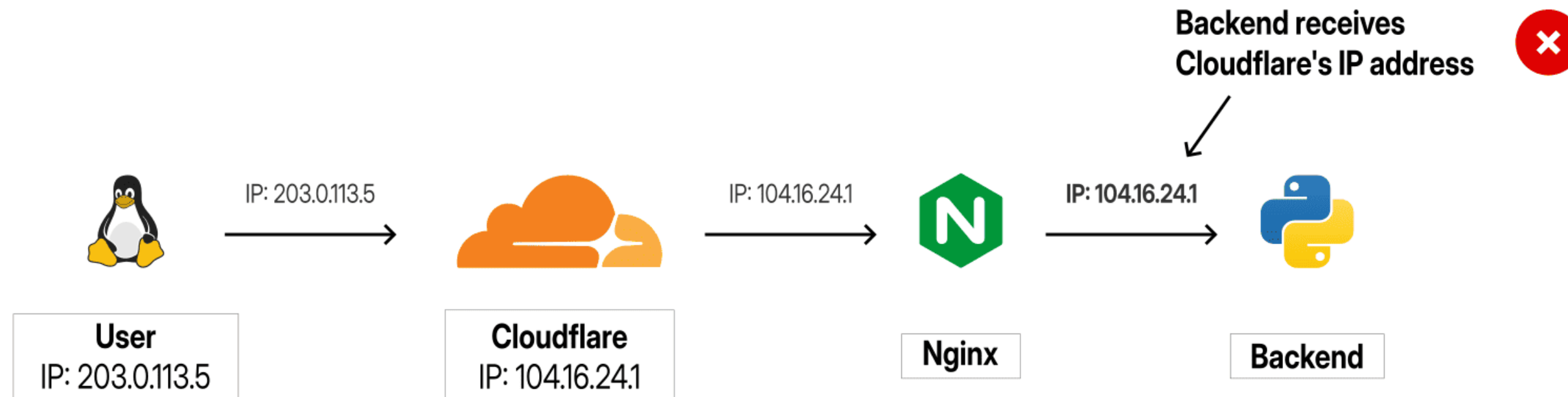


# The real IP problem





# The real IP problem



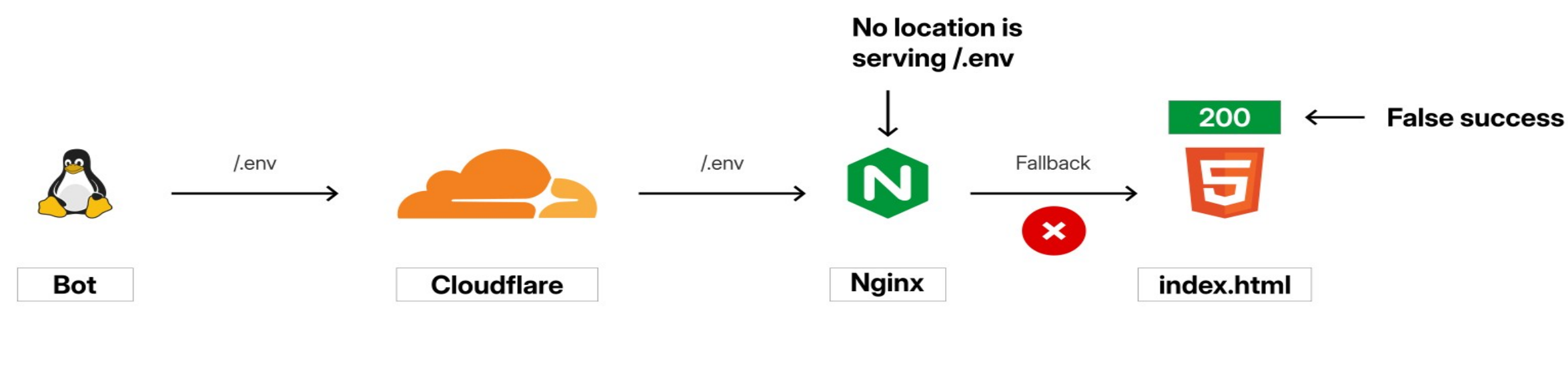
# **Module 4: Global delivery and security**

## **Vulnerability scanning**

# Testing your defenses

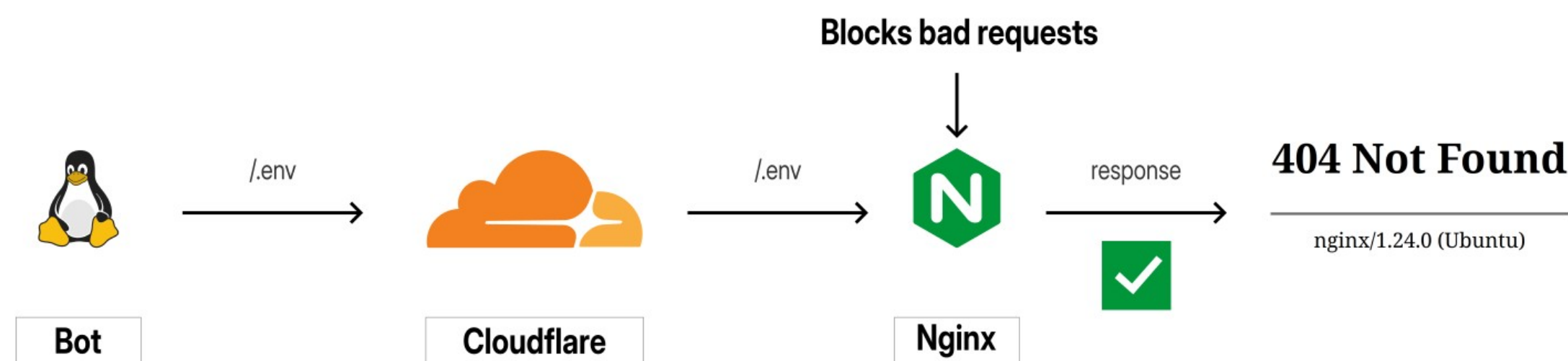
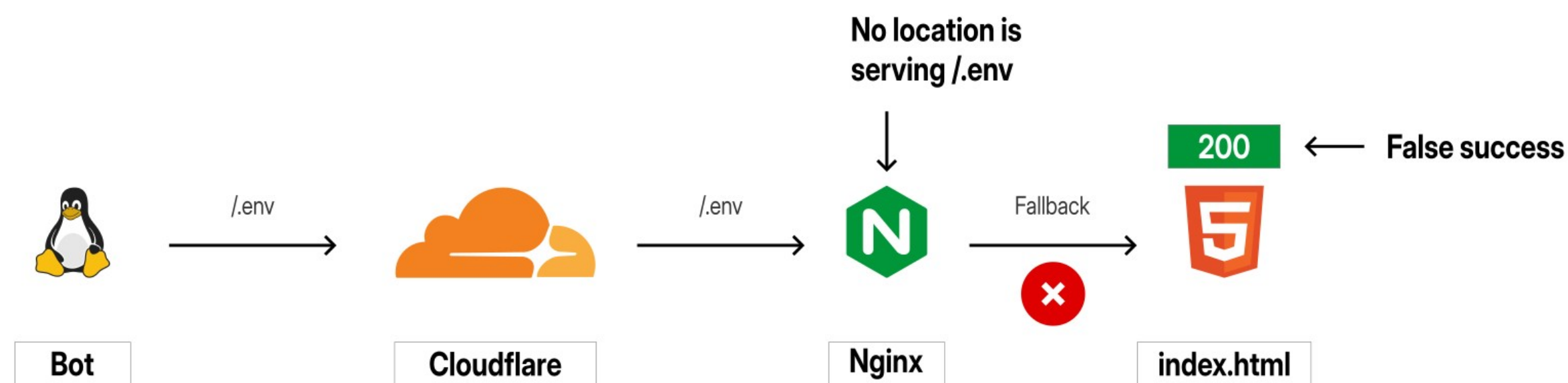
- Understand and fix the **Soft 404** vulnerability.
- Configure Nginx to instantly **block malicious bots**.
- Use nmap to **scan your firewall** from the outside.

# Fixing the SPA trap (Soft 404s)





# Fixing the SPA trap (Soft 404s)



# **Module 5: The automation pipeline**

**Local hygiene**

# Securing your local workflow

- Protect the **main** branch using **GitHub Rulesets**.
- Automate local checks with **pre-commit**.
- Lint and format Python using **Ruff**.
- Lint and format Vue.js using **ESLint & Prettier**.